Compte rendu – Site internet de SPOUDAZO Traiteur

Projet client · Kinshasa, République Démocratique du Congo

KANSIETOKO Yuung Addi

Juillet – août 2026 · Next.js / React / TypeScript

1. Contexte et objectif

SPOUDAZO Traiteur est un service traiteur de Kinshasa, en République Démocratique du Congo, fondé en 2010. Il travaille pour des ministères, des ambassades, des organisations internationales et des entreprises, et il exploite un Bar & Restaurant dans l'enceinte de la FIKIN. Le client m'a remis un cahier des charges en juillet 2026 : un site vitrine qui présente l'entreprise, ses prestations, sa carte et ses références, et qui permet de demander un devis en ligne.

C'est un projet pour un vrai client. J'ai mené la conception, le développement, la sécurité et toute la documentation, jusqu'à la version 1.6 livrée fin août 2026.

Élément	Choix
Framework	Next.js 15 (App Router), React 19, TypeScript
Styles	Tailwind CSS 4
Base de données des avis	Supabase (PostgreSQL), avec sécurité au niveau des lignes
Hébergement prévu	Vercel (HTTPS automatique), nom de domaine à acheter par le client
Code source	Dépôt GitHub privé (projet client)

2. Le choix technique

J'ai comparé trois solutions selon les trois priorités du client : la sécurité, la facilité d'entretien et la possibilité de faire évoluer le site.

- **WordPress**, écarté : page de connexion, base MySQL et extensions à mettre à jour forment une grande surface d'attaque. C'est par là que passent la plupart des piratages de sites de petites entreprises.
- **Un site HTML écrit à la main**, écarté : le contenu serait recopié sur chaque page, et le site ne pourrait pas évoluer.
- **Wix ou Squarespace**, écartés : abonnement mensuel, personnalisation limitée, aucune maîtrise du code.

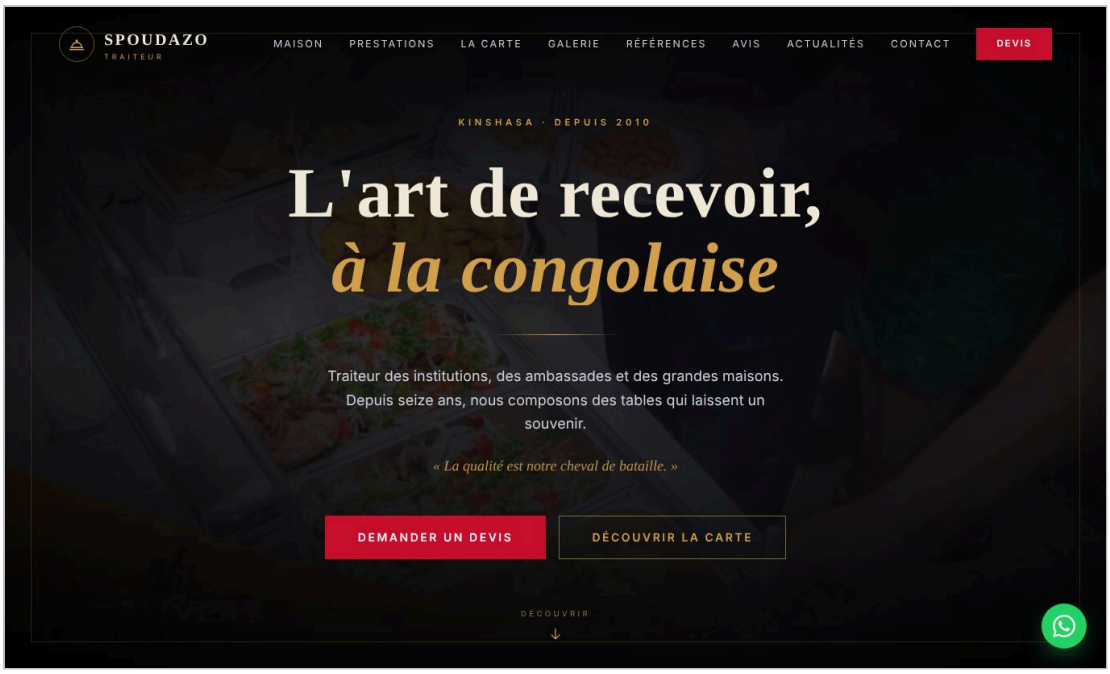
J'ai retenu Next.js avec génération statique : les pages sont fabriquées une fois, au moment de la compilation. Sans base de données ni serveur exposé pour les pages, l'injection SQL ou la prise de contrôle d'une page d'administration n'ont aucune prise.

3. Le contenu et les pages

J'ai saisi tout le contenu du cahier des charges : 11 catégories de carte avec plus de 100 articles et leurs prix en francs congolais, 9 formules traiteur, 5 catégories de références, les valeurs et la direction. Ce

contenu est rangé dans 4 fichiers de données (`src/data/`), séparés du code d'affichage : changer un prix demande de modifier une seule ligne, sans toucher au design.

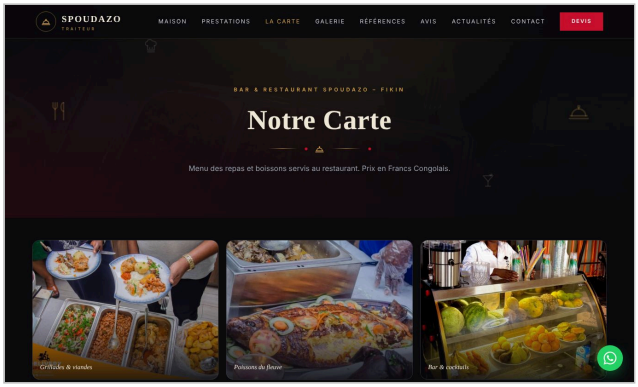
Le site compte 11 pages : accueil, à propos, services, carte, références, galerie, actualités, avis, dépôt d'un avis, devis et contact. La compilation produit 16 routes sans aucune erreur.



Page d'accueil : bandeau plein écran et appel au devis



Les prestations



La carte du Bar & Restaurant

4. Le design, en trois versions

La première version reprenait les couleurs de la charte (rouge, blanc, noir, bleu) sur un fond clair. Retour du client : trop fade. J'ai ajouté des animations et des icônes de restauration. Deuxième retour : pas assez chic. J'ai alors refait le site sur un parti pris de « restaurant étoilé » : fond noir profond, accents dorés, titres en Playfair Display, photos pleine largeur, le rouge de la charte gardé pour les boutons d'action. Les 10 animations respectent le réglage « réduire les animations » du système, pour les personnes sensibles au mouvement.

Chaque photo a une illustration dessinée en dessous : si l'image ne charge pas, l'illustration prend sa place, et le site n'affiche jamais de zone vide. C'est un vrai enjeu avec la qualité du réseau en RDC. Le client m'a ensuite fourni 45 photographies réelles, intégrées dans la galerie par catégories.



La galerie, classée par catégories



L'accueil sur téléphone

5. Les avis clients et le QR code

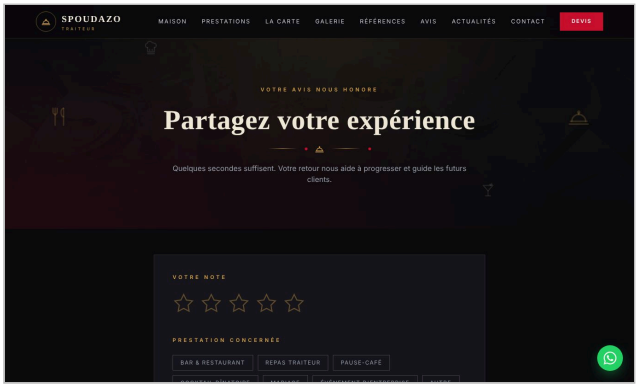
J'ai ajouté, en plus du cahier des charges, un système d'avis : le client scanne un QR code posé sur la table, donne une note en étoiles et un témoignage. L'avis part en attente, et il n'est publié qu'après validation par SPOUDAZO. Les avis critiques sincères sont publiés aussi : un profil uniquement composé de cinq étoiles inspire la méfiance.

L'interface qui reçoit les avis applique 7 contrôles : modération obligatoire, champ piège invisible pour les robots, délai minimal de 3 secondes, 3 avis par heure et 3 par jour au maximum depuis une même adresse, suppression des balises HTML et des liens. L'adresse IP n'est jamais stockée en clair, seulement son empreinte SHA-256 salée. J'ai testé chaque contrôle : note hors plage, commentaire trop court, envoi trop rapide, quatrième avis de suite et mauvaise méthode HTTP sont tous refusés.

Tant que la base Supabase n'est pas configurée, le site affiche des avis de démonstration : il peut donc être mis en ligne tout de suite. J'ai aussi généré les QR codes et deux supports prêts à imprimer, un chevalet de table et une affiche A5.



La page des avis : note moyenne et répartition



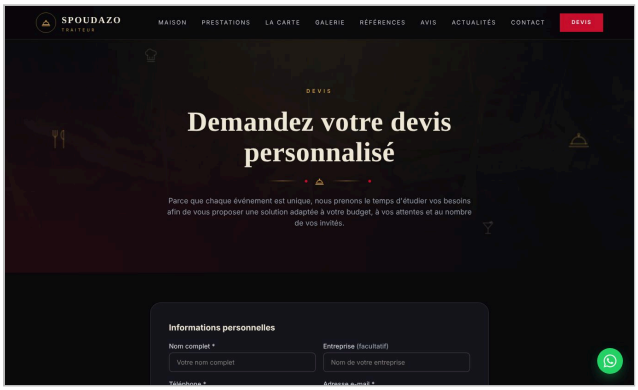
Le dépôt d'un avis, cible du QR code

6. La sécurité

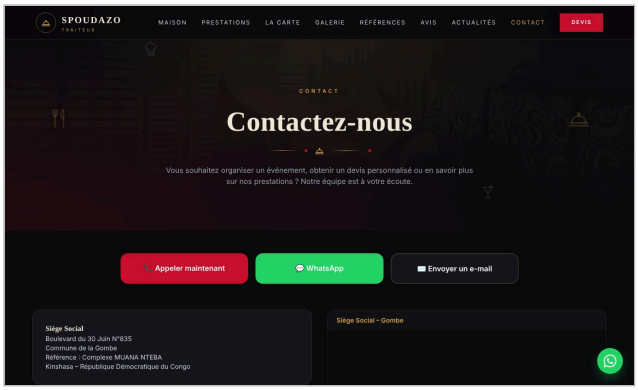
Le serveur envoie six en-têtes de sécurité, que j'ai vérifiés sur la version compilée :

- `Content-Security-Policy` : la liste des seules sources autorisées pour les scripts, les styles, les images et les cadres.
- `Strict-Transport-Security` : HTTPS obligatoire pendant deux ans.
- `X-Frame-Options: DENY` : le site ne peut pas être affiché dans un cadre sur un autre site (protection contre le clickjacking).
- `X-Content-Type-Options`, `Referrer-Policy` et `Permissions-Policy` (caméra, micro et géolocalisation coupés).

J'ai aussi pris une décision de sécurité côté métier : le cahier des charges demandait d'afficher les coordonnées bancaires, et je ne les ai pas publiées. Un escroc pourrait copier la page, remplacer le numéro de compte et détourner les paiements. Ces coordonnées doivent figurer seulement sur les factures. Les clés secrètes restent côté serveur et ne sont jamais dans le code.



Le formulaire de demande de devis



Contact : appel, WhatsApp et e-mail en un geste

7. La documentation

J'ai rédigé 8 documents pour que le client et un futur développeur puissent reprendre le projet : dossier de projet avec les décisions d'architecture, journal des versions, sécurité, hébergement, maintenance, avis et QR codes, guide de mise en ligne sans jargon, et état d'avancement.

8. Bilan et suite

Le développement, la sécurité et la documentation sont terminés. Il reste la mise en ligne, qui dépend du client : créer les comptes GitHub et Vercel à son nom, avec la double authentification, puis acheter le nom de domaine. Les QR codes seront régénérés vers l'adresse définitive avant toute impression. La phase 2 prévoit un espace administrateur pour que le client modifie lui-même la carte, les photos et les actualités.

Ce projet m'a appris à travailler avec un vrai client : comprendre un cahier des charges, justifier un choix technique, intégrer des retours successifs sur le design, et refuser une demande quand elle met le client en danger, en expliquant pourquoi.